

M04 / ASSESSMENT

M04 Module Revision Test

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
Rules	3
T01 - DE04-01 Python setup and first program	3
T02 - DE04-02 Variables, data types and operators	3
T03 - DE04-03 Strings and formatting	3
T04 - DE04-04 Conditions and boolean logic	4
T05 - DE04-05 Lists, tuples, sets and dictionaries	4
T06 - DE04-06 Loops and iteration	4
T07 - DE04-07 Functions and scope	4
T08 - DE04-08 Modules, imports and project structure	4
T09 - DE04-09 Exceptions and debugging basics	4
T10 - DE04-10 Files, paths, CSV and JSON I/O	5
T11 - DE04-11 pandas foundations: load, inspect, filter and aggregate	5
T12 - DE04-12 Practical Big-O intuition	5
T13 - DE04-13 Coding drills: lists and strings	5
T14 - DE04-14 Coding drills: dictionaries and sets	6
T15 - DE04-15 Stack, queue, search and sort basics	6
T16 - DE04-16 Integrated Python checkpoint and reproducible script habits	6

Read this first

MODULE REVISION TEST

Save your answers before opening the protected answer guide. No forced timer. First prove the competency untimed; later you may record interview-style time if useful. This test does not replace a protected Gate.

Coverage: 16 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

Rules

Work with learner book, review book, answer guide and AI solving closed. Save one answer/evidence item for every lesson. If a task needs a PC, write your prediction first, then run it. Mark PASS only when the result and your explanation agree.

T01 - DE04-01 | Python setup and first program

If you needed the Explained Review, close it before retrying. Create another file that prints a different label and calculates $9 * 6$. Introduce another syntax error deliberately, explain the final exception line before changing code, then repair it without reopening the review.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T02 - DE04-02 | Variables, data types and operators

Use `shipment_id="S009"`, `cartons=7`, `unit_cost=425.5`, `delivered=False`. Calculate shipment cost and:

`needs_review = cost > 2500 and not delivered`

Predict both outputs before running. Then explain why not delivered is meaningful only because delivered is a boolean contract rather than a string such as "False".

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T03 - DE04-03 | Strings and formatting

Given " Pen Blue ", "pen blue" (two internal spaces) and "PEN BLUE", normalize only surrounding spaces and case. Compare results. Do not silently collapse internal spaces. Explain what additional business rule would be needed before deciding those labels are equivalent.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T04 - DE04-04 | Conditions and boolean logic

A shipment is escalate only when `delayed_days >= 2` and `priority_customer` is `True`. Test 1, 2 and 3 delayed days for both customer flags, plus one `priority_customer=None` case. State what your code does with unknown priority and why.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T05 - DE04-05 | Lists, tuples, sets and dictionaries

Represent warehouse zones and supervisors, a unique set of failed shipment IDs, a processing sequence, and a fixed (`country_code`, `year`) pair. Add a case where one zone legitimately has two supervisors and redesign the dictionary value rather than silently overwriting one name.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T06 - DE04-06 | Loops and iteration

Given shipment weights `[12.5, -1, 0, 8.0, 15.2]`, create accepted and rejected lists, total only positive weights, and leave the source list intact. Explain why removing bad values from the same list during iteration is a weaker design.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T07 - DE04-07 | Functions and scope

Write `shipment_cost(shipment_id, cartons, cost_per_carton)` returning a dict with ID and cost. Test three cases including zero cartons. Then extend it to accept an explicit currency

argument rather than reading a hidden global. Explain which inputs are measures and which are identifiers/labels.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T08 - DE04-08 | Modules, imports and project structure

Create `shipment_helpers.py` with `shipment_value(cartons, unit_cost)` and `normalize_hub(text)`. Create `run_shipment.py` with `main()`. Prove the helper can be imported without printing or starting the workflow. Explain which file owns reusable logic and which owns run orchestration.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T09 - DE04-09 | Exceptions and debugging basics

Write `safe_positive_int(raw_value)` that either returns a positive integer or raises a clear `ValueError` for non-integer text, zero and negative values. Test "4", "0", "-2" and "four". Explain why parsing and business validation are two different checks.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T10 - DE04-10 | Files, paths, CSV and JSON I/O

Create a fresh manifest for M04_Retest_Shipments.csv, read its CSV header with the standard csv module and validate these required fields: shipment_id, hub, status, cartons, unit_cost, coordinator. Add a deliberately wrong required field name and preserve the resulting missing-column error before repairing it.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T11 - DE04-11 | pandas foundations: load, inspect, filter and aggregate

Use M04_Retest_Shipments.csv. Calculate shipment_value = cartons * unit_cost; filter Delivered; aggregate Delivered value by hub; show top 3 Delivered shipments with a deterministic second key; count missing coordinator values over the population you explicitly name. Do not look at the review controls until your query/code is saved.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T12 - DE04-12 | Practical Big-O intuition

Suppose each order must attach a region label from a customer mapping. Compare scanning every customer for each order with building one customer_id -> region dictionary first. State approximate build cost, lookup cost and total-growth intuition. Do not claim dictionary lookup is "always literally one CPU operation."

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T13 - DE04-13 | Coding drills: lists and strings

In a fresh file, implement:

```
normalize_words(text)
```

It should strip surrounding whitespace, split on runs of whitespace, lowercase tokens and return a list of tokens. Test blank text, one word and text with multiple spaces. This is deliberately different from reverse_words: the output type changes and no reversing is requested.

Continuous interview habit From this lesson onward, add one 15-20 minute closed-book Python micro-drill to study days. Stop at the time limit, preserve the attempt, review the reasoning, then redo a changed problem later. Do not spend hours on advanced competitive programming; the aim is reliable basic problem solving for DE screening and debugging.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T14 - DE04-14 | Coding drills: dictionaries and sets

Write `duplicate_keys(records)` where each record has `order_id` and `line_no`. Return composite keys that appear more than once, without reporting the same duplicate key multiple times. Test no duplicates, one duplicate pair and two different duplicate pairs. Explain why checking only `order_id` would incorrectly flag legitimate multiple lines.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T15 - DE04-15 | Stack, queue, search and sort basics

Use tasks `['ingest','validate','publish']` to contrast stack and queue removal order. Search target 12 inside `[4,12,7,3]` with linear search. Create a sorted copy before any binary-search attempt, and explain the extra sorting step. Add an absent-target test.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T16 - DE04-16 | Integrated Python checkpoint and reproducible script habits

Only if the review/repair route assigns it, close all answer material and implement `M04_Fresh_Checkpoint_TODO.py` against `M04_Retest_Shipments.csv`. This changed scenario uses shipment/hub language, Delivered filtering and different controls. It is not a word-for-word clone of retail orders.

You must validate schema; calculate `shipment_value`; aggregate Delivered value by hub; produce deterministic top 3; count missing coordinator values over a declared population; write/read JSON; prove one failure; and explain input/report grain. The fresh review remains closed until that attempt is saved.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY